Introduction to PLINK and R

Part 1: PLINK

> **Kernel:** run the code cells in this section with the **Bash** kernel.

Running PLINK

Here we are running PLINK in a Juypter notebook. These commands can also be run on a command line. In both cases, additional arguments ("options", "flags") specify exactly what PLINK should do. All arguments are documented on the PLINK website.

Data overview

In this exercise toy example will be used on diastolic blood pressure and the genotypes for 20 SNP markers, which are in a file in PLINK format. This exercise uses four files:

- `dbp.qt.ped` — pedigree file with family, sex, the quantitative trait (diastolic blood pressure), and genotypes
- `dbp.cc.ped` — pedigree file with family, sex, the dichotomized trait (affected yes/no based on blood pressure), and genotypes
- `dbp.map` — map file for the SNP markers (chromosome, SNP ID, genetic distance, position)
- `dbp.age.pheno` — covariate file containing the age of each individual

References

- Chang CC, Chow CC, Tellier LCAM, Vattikuti S, Purcell SM, Lee JJ (2015) Second-generation PLINK: rising to the challenge of larger and richer datasets. GigaScience, 4.
- Purcell S, Neale B, Todd-Brown K, Thomas L, Ferreira MAR, Bender D, Maller J, Sklar P, de Bakker PIW, Daly MJ & Sham PC (2007) PLINK: a toolset for whole-genome association and population-based linkage analysis. *American Journal of Human Genetics*, 81:559-575.

I. Preview Input Files

In this section we'll take a look at the contents of these files. Make sure you understand the meaning of each column.

In [1]:
```
# Preview dbp.qt.ped
head ./data/dbp.qt.ped
```

```
4928    1       0       0       1       85.51   2       2       1       1       1
1       0       0       1       1       1       2       1       1       2       2
1       1       1       1       1       2       1       2       1       2       1
2       1       2       2       2       1       1       1       2       1       2
1       2
1838    1       0       0       1       84.51   1       1       1       1       2
2       2       2       2       2       1       1       1       1       1       1
2       2       2       2       2       2       1       1       1       1       2
2       2       2       2       2       1       1       1       1       2       2
2       2
2450    1       0       0       1       84.3    1       1       1       1       2
2       2       2       2       2       0       0       1       1       1       1
2       2       2       2       2       2       1       2       1       2       2
2       1       2       2       2       1       1       1       2       1       2
1       2
647     1       0       0       2       89.14   2       2       2       2       1
2       1       2       2       2       1       1       1       2       1       2
2       2       1       2       1       1       1       2       1       2       1
1       1       2       2       2       1       2       1       2       2       2
1       2
2772    1       0       0       1       90.39   1       2       1       1       1
2       0       0       1       2       1       1       1       1       1       2
1       2       1       2       1       1       1       1       1       1       1
1       2       2       2       2       1       1       1       1       2       2
2       2
148     1       0       0       2       86.24   2       2       1       1       1
1       2       2       1       1       0       0       1       1       2       2
1       1       1       1       1       1       1       2       1       2       1
1       1       2       2       2       1       2       1       2       2       2
1       2
1       1       0       0       1       82.33   1       2       1       2       2
2       2       2       2       2       1       1       1       2       1       1
2       2       1       2       1       1       1       1       1       2       1
2       2       2       1       2       1       1       1       1       1       2
1       2
1696    1       0       0       2       82.64   1       2       1       2       1
2       2       2       1       2       1       2       1       2       1       2
1       1       1       1       1       2       1       2       1       2       1
1       2       2       2       2       1       2       1       2       2       2
2       2
890     1       0       0       1       82.85   1       2       1       1       1
2       0       0       1       2       1       1       1       1       1       2
1       2       1       2       1       1       1       2       1       1       1
2       1       2       1       2       1       2       1       1       1       2
2       2
1832    1       0       0       1       90.97   1       2       1       1       1
2       2       2       1       2       1       1       1       1       1       2
1       2       1       2       1       2       1       1       1       1       1
2       2       2       2       2       1       1       1       1       2       2
2       2
```

In [2]:
```
# Preview dbp.cc.ped
```

```
head ./data/dbp.cc.ped
```

```
4928 1    0    0    1    2    2    2    1    1    1    1    0    0    1    1
1    2    1    1    2    2    1    1    1    1    1    2    1    2    1    2
1    2    1    2    2    2    1    1    1    2    1    2    1    2
1838 1    0    0    1    2    1    1    1    1    2    2    2    2    2    2
1    1    1    1    1    1    2    2    2    2    2    2    1    1    1    1
2    2    2    2    2    2    1    1    1    1    2    2    2    2
2450 1    0    0    1    2    1    1    1    1    2    2    2    2    2    2
0    0    1    1    1    1    2    2    2    2    2    2    1    2    1    2
2    2    1    2    2    2    1    1    1    2    1    2    1    2
647  1    0    0    2    2    2    2    2    2    1    2    1    2    2    2
1    1    1    2    1    2    2    2    1    2    1    1    1    2    1    2
1    1    1    2    2    2    1    2    1    2    2    2    1    2
2772 1    0    0    1    2    1    2    1    1    1    2    0    0    1    2
1    1    1    1    1    2    1    2    1    2    1    1    1    1    1    1
1    1    2    2    2    2    1    1    1    1    2    2    2    2
148  1    0    0    2    2    2    2    1    1    1    1    2    2    1    1
0    0    1    1    2    2    1    1    1    1    1    1    1    2    1    2
1    1    1    2    2    2    1    2    1    2    2    2    1    2
1    1    0    0    1    2    1    2    1    2    2    2    2    2    2    2
1    1    1    2    1    1    2    2    1    2    1    1    1    1    1    2
1    2    2    2    1    2    1    1    1    1    1    2    1    2
1696 1    0    0    2    2    1    2    1    2    1    2    2    2    1    2
1    2    1    2    1    2    1    1    1    1    1    2    1    2    1    2
1    1    2    2    2    2    1    2    1    2    2    2    2    2
890  1    0    0    1    2    1    2    1    1    1    2    0    0    1    2
1    1    1    1    1    2    1    2    1    2    1    1    1    2    1    1
1    2    1    2    1    2    1    2    1    1    1    2    2    2
1832 1    0    0    1    2    1    2    1    1    1    2    2    2    1    2
1    1    1    1    1    2    1    2    1    2    1    2    1    1    1    1
1    2    2    2    2    2    1    1    1    1    2    2    2    2
```

In [3]:
```
# Preview dbp.map
head ./data/dbp.map
```

```
11  rs1101   0    1021
11  rs1102   0    3886
11  rs1103   0    15023
11  rs1104   0    15788
11  rs1105   0    21702
11  rs1106   0    23505
11  rs1107   0    23508
11  rs1108   0    28769
11  rs1109   0    31385
11  rs1110   0    33198
```

In [4]:
```
# Preview dbp.age.pheno
head ./data/dbp.age.pheno
```

```
4928 1    66
1838 1    67
2450 1    89
647  1    36
2772 1    54
148  1    55
1    1    63
1696 1    83
890  1    80
1832 1    76
```

In [5]:
```
# Set the output directory for this exercise
output_dir="./output/intro_plink_R"
```

II. Missing Data and Filtering

Variables with too many missing values can bias a statistical analysis and lead to spurious results. We'll use PLINK to assess the extent of missing values in the data set, then filter out variables and samples with too many missing observations.

PLINK requires the data set to process as its first argument, specified with the `--ped` and `--map` options. `dbp.map` is already in the standard four-column format (chromosome, SNP ID, genetic distance, position), and this version of PLINK auto-detects the number of columns, so no extra flag is needed for it. First, let's assess the proportion of missing values for each marker and each sample.

> **Note:** All arguments of a PLINK call must go on a single line — anything after a line break is ignored. A trailing backslash (`\`) suppresses the line break and lets a single PLINK call span multiple lines, e.g.:
>
> ```
> plink --ped dbp.cc.ped \
>     --map dbp.map \
>     --missing
> ```

In [6]:
```
# Run plink command
plink  --ped ./data/dbp.cc.ped  --map ./data/dbp.map  --missing --out ${outp
```

```
PLINK v1.9.0-b.8 64-bit (22 Oct 2024)             cog-genomics.org/plink/1.
9/
(C) 2005-2024 Shaun Purcell, Christopher Chang   GNU General Public License
v3
Logging to ./output/intro_plink_R/plink.log.
Options in effect:
  --map ./data/dbp.map
  --missing
  --out ./output/intro_plink_R/plink
  --ped ./data/dbp.cc.ped

7836 MB RAM detected; reserving 3918 MB for main workspace.
.ped scan complete (for binary autoconversion).
Performing single-pass .bed write (20 variants, 600 people).
--file: ./output/intro_plink_R/plink-temporary.bed +
./output/intro_plink_R/plink-temporary.bim +
./output/intro_plink_R/plink-temporary.fam written.
20 variants loaded from .bim file.
600 people (329 males, 271 females) loaded from .fam.
600 phenotype values loaded from .fam.
Using 1 thread (no multithreaded calculations invoked).
Before main variant filters, 600 founders and 0 nonfounders present.
Calculating allele frequencies... done.
Total genotyping rate is 0.988333.
--missing: Sample missing data report written to
./output/intro_plink_R/plink.imiss, and variant-based missing data report
written to ./output/intro_plink_R/plink.lmiss.
```

PLINK has created three files: `plink.log` contains all screen output, while `plink.imiss` and `plink.lmiss` contain the proportion of missing values for each sample and marker, respectively. Preview all three files below.

In [7]:
```
# Preview output file - plink.log
head ${output_dir}/plink.log
```

```
PLINK v1.9.0-b.8 64-bit (22 Oct 2024)
Options in effect:
  --map ./data/dbp.map
  --missing
  --out ./output/intro_plink_R/plink
  --ped ./data/dbp.cc.ped

Hostname: 39213c7743fc
Working directory: /home/jupyter
Start time: Thu Sep 10 06:50:16 2026
```

In [8]:
```
# Preview output file - plink.imiss
head ${output_dir}/plink.imiss
```

```
 FID  IID MISS_PHENO   N_MISS   N_GENO   F_MISS
4928    1          N        1       20     0.05
1838    1          N        0       20        0
2450    1          N        1       20     0.05
 647    1          N        0       20        0
2772    1          N        1       20     0.05
 148    1          N        1       20     0.05
   1    1          N        0       20        0
1696    1          N        0       20        0
 890    1          N        1       20     0.05
```

In [9]:
```
# Preview output file - plink.lmiss
head ${output_dir}/plink.lmiss
```

```
CHR      SNP   N_MISS   N_GENO   F_MISS
 11   rs1101        0      600        0
 11   rs1102        0      600        0
 11   rs1103        0      600        0
 11   rs1104       92      600   0.1533
 11   rs1105        0      600        0
 11   rs1106       48      600     0.08
 11   rs1107        0      600        0
 11   rs1108        0      600        0
 11   rs1109        0      600        0
```

Next, we'll exclude samples with more than 10% missing genotypes (`--mind 0.10`) and markers with more than 5% missing (`--geno 0.05`). We'll write this filtered data set to new files, `cleaned.ped` and `cleaned.map` , using `--recode` and `--out` . Further quality checks and analyses can then be based on this cleaned data set.

In [10]:
```
# Run the plink command
plink  --ped ./data/dbp.cc.ped  --map ./data/dbp.map  --mind 0.10  --geno 0.
       --recode  --out ${output_dir}/cleaned
```

```
PLINK v1.9.0-b.8 64-bit (22 Oct 2024)             cog-genomics.org/plink/1.
9/
(C) 2005-2024 Shaun Purcell, Christopher Chang   GNU General Public License
v3
Logging to ./output/intro_plink_R/cleaned.log.
Options in effect:
  --geno 0.05
  --map ./data/dbp.map
  --mind 0.10
  --out ./output/intro_plink_R/cleaned
  --ped ./data/dbp.cc.ped
  --recode

7836 MB RAM detected; reserving 3918 MB for main workspace.
.ped scan complete (for binary autoconversion).
Performing single-pass .bed write (20 variants, 600 people).
--file: ./output/intro_plink_R/cleaned-temporary.bed +
./output/intro_plink_R/cleaned-temporary.bim +
./output/intro_plink_R/cleaned-temporary.fam written.
20 variants loaded from .bim file.
600 people (329 males, 271 females) loaded from .fam.
600 phenotype values loaded from .fam.
0 people removed due to missing genotype data (--mind).
Using 1 thread (no multithreaded calculations invoked).
Before main variant filters, 600 founders and 0 nonfounders present.
Calculating allele frequencies... done.
Total genotyping rate is 0.988333.
2 variants removed due to missing genotype data (--geno).
18 variants and 600 people pass filters and QC.
Among remaining phenotypes, 300 are cases and 300 are controls.
--recode ped to ./output/intro_plink_R/cleaned.ped +
./output/intro_plink_R/cleaned.map ... done.
```

PLINK has created three files: a log file, `cleaned.log` (from the `--out` flag), and the two data files `cleaned.ped` and `cleaned.map`. Two markers with too many missing values (`rs1104` and `rs1106`) have been excluded. Preview all three files below.

In [11]:
```
# Preview output file - cleaned.log
head ${output_dir}/cleaned.log
```

```
PLINK v1.9.0-b.8 64-bit (22 Oct 2024)
Options in effect:
  --geno 0.05
  --map ./data/dbp.map
  --mind 0.10
  --out ./output/intro_plink_R/cleaned
  --ped ./data/dbp.cc.ped
  --recode

Hostname: 39213c7743fc
```

In [12]:
```
# Preview output file - cleaned.ped
head ${output_dir}/cleaned.ped
```

```
4928 1 0 0 1 2 2 2 1 1 1 1 1 1 1 2 2 1 1 1 2 1 2 1 2 1 2 1 1 2 2 2 1 1 2
1 1 2 1 2
1838 1 0 0 1 2 1 1 1 1 2 2 2 2 1 1 1 1 2 2 2 2 2 2 1 1 1 1 2 2 2 2 2 2 1 1 1
1 2 2 2 2
2450 1 0 0 1 2 1 1 1 1 2 2 2 2 1 1 1 1 2 2 2 2 2 2 2 1 2 1 2 2 1 2 2 2 1 1 2
1 1 2 1 2
647 1 0 0 2 2 2 2 2 2 2 1 2 2 2 1 2 1 2 2 1 2 1 1 2 1 2 1 1 1 1 2 2 2 2 1 2
1 2 2 1 2
2772 1 0 0 1 2 1 2 1 1 2 1 1 2 1 1 2 1 1 2 1 2 1 1 1 1 1 1 1 2 2 2 2 1 1 1
1 2 2 2 2
148 1 0 0 2 2 2 2 1 1 1 1 1 1 1 2 2 1 1 1 1 1 2 1 2 1 1 1 1 2 2 2 2 1 2
1 2 2 1 2
1 1 0 0 1 2 1 2 2 1 2 2 2 2 2 1 1 1 2 2 1 2 1 1 1 1 2 1 2 1 2 2 1 2 1 1 1 1
1 2 1 2
1696 1 0 0 2 2 1 2 2 1 2 1 1 2 2 1 2 1 1 1 1 1 2 1 2 1 2 1 1 1 2 2 2 2 2 1 2
1 2 2 2 2
890 1 0 0 1 2 1 2 1 1 2 1 1 2 1 1 2 1 1 2 1 2 1 1 2 1 1 1 2 1 1 2 1 2 2 1 1
1 1 2 2 2
1832 1 0 0 1 2 1 2 1 1 2 1 1 2 1 1 2 1 1 2 1 2 2 1 1 1 1 1 2 1 2 2 2 2 1 1 1
1 2 2 2 2
```

In [13]:
```
# Preview output file - cleaned.map
head ${output_dir}/cleaned.map
```

```
11      rs1101  0       1021
11      rs1102  0       3886
11      rs1103  0       15023
11      rs1105  0       21702
11      rs1107  0       23508
11      rs1108  0       28769
11      rs1109  0       31385
11      rs1110  0       33198
11      rs1111  0       1245388
11      rs1112  0       1245604
```

Next, we'll use this filtered data set to estimate the minor allele frequencies (MAF) of the markers with the `--freq` flag. This step estimates the MAFs but does not filter on a minimum frequency (use `--maf` for that). The resulting file, `cleaned.frq`, contains the frequency estimates.

Note: MAF is always ≤ 0.50 — PLINK automatically assigns the reference allele so that `A2` is the major (more frequent, "baseline") allele and `A1` is the minor (less frequent, "risk") allele. This automatic allele flipping applies throughout many PLINK analyses, including association testing, so always check which allele is the baseline and which is the minor/risk allele in your results. For marker `rs1101`, for example, allele `2` is the major allele `A2` and allele `1` is the minor allele `A1`. To avoid automatic allele flipping altogether, use the `--keep-allele-order` flag throughout.

In [14]:
```
# Run the plink command
plink  --ped ${output_dir}/cleaned.ped  --map ${output_dir}/cleaned.map  --f
```

```
PLINK v1.9.0-b.8 64-bit (22 Oct 2024)             cog-genomics.org/plink/1.
9/
(C) 2005-2024 Shaun Purcell, Christopher Chang   GNU General Public License
v3
Logging to ./output/intro_plink_R/cleaned.log.
Options in effect:
  --freq
  --map ./output/intro_plink_R/cleaned.map
  --out ./output/intro_plink_R/cleaned
  --ped ./output/intro_plink_R/cleaned.ped

7836 MB RAM detected; reserving 3918 MB for main workspace.
.ped scan complete (for binary autoconversion).
Performing single-pass .bed write (18 variants, 600 people).
--file: ./output/intro_plink_R/cleaned-temporary.bed +
./output/intro_plink_R/cleaned-temporary.bim +
./output/intro_plink_R/cleaned-temporary.fam written.
18 variants loaded from .bim file.
600 people (329 males, 271 females) loaded from .fam.
600 phenotype values loaded from .fam.
Using 1 thread (no multithreaded calculations invoked).
Before main variant filters, 600 founders and 0 nonfounders present.
Calculating allele frequencies... done.
Total genotyping rate is exactly 1.
--freq: Allele frequencies (founders only) written to
./output/intro_plink_R/cleaned.frq .
```

In [15]:
```
# Preview the output - cleaned.frq
head ${output_dir}/cleaned.frq
```

```
CHR      SNP   A1   A2          MAF  NCHROBS
 11   rs1101    1    2       0.4508     1200
 11   rs1102    2    1       0.2642     1200
 11   rs1103    2    1       0.4675     1200
 11   rs1105    1    2       0.4558     1200
 11   rs1107    2    1       0.1525     1200
 11   rs1108    2    1         0.48     1200
 11   rs1109    1    2       0.4425     1200
 11   rs1110    1    2       0.4558     1200
 11   rs1111    2    1        0.435     1200
```

Now let's check for deviations from Hardy-Weinberg equilibrium (HWE).

In [16]:
```
# Run the plink command
plink  --ped ${output_dir}/cleaned.ped  --map ${output_dir}/cleaned.map  --h
```

```
PLINK v1.9.0-b.8 64-bit (22 Oct 2024)             cog-genomics.org/plink/1.
9/
(C) 2005-2024 Shaun Purcell, Christopher Chang   GNU General Public License
v3
Logging to ./output/intro_plink_R/cleaned.log.
Options in effect:
  --hardy
  --map ./output/intro_plink_R/cleaned.map
  --out ./output/intro_plink_R/cleaned
  --ped ./output/intro_plink_R/cleaned.ped

7836 MB RAM detected; reserving 3918 MB for main workspace.
.ped scan complete (for binary autoconversion).
Performing single-pass .bed write (18 variants, 600 people).
--file: ./output/intro_plink_R/cleaned-temporary.bed +
./output/intro_plink_R/cleaned-temporary.bim +
./output/intro_plink_R/cleaned-temporary.fam written.
18 variants loaded from .bim file.
600 people (329 males, 271 females) loaded from .fam.
600 phenotype values loaded from .fam.
Using 1 thread (no multithreaded calculations invoked).
Before main variant filters, 600 founders and 0 nonfounders present.
Calculating allele frequencies... done.
Total genotyping rate is exactly 1.
--hardy: Writing Hardy-Weinberg report (founders only) to
./output/intro_plink_R/cleaned.hwe ... done.
```

This step tests for deviations but doesn't filter on a P-value threshold (use `--hwe` for that). The output file `cleaned.hwe` has three result lines per marker: one each for cases, controls, and the complete sample. Each line contains the baseline allele (`A2`), the observed genotype counts (`GENO`), the observed and expected heterozygote frequencies (`O(HET)` and `E(HET)`), and the corresponding P-value (`P`). Preview the resulting file, `cleaned.hwe`, below.

In [17]:
```
# Preview the output - cleaned.hwe
head ${output_dir}/cleaned.hwe
```

```
 CHR      SNP     TEST   A1   A2                 GENO   O(HET)   E(HET)
P
  11   rs1101      ALL    1    2          115/311/174   0.5183   0.4952
0.2838
  11   rs1101      AFF    1    2            54/159/87     0.53    0.494
0.2426
  11   rs1101    UNAFF    1    2            61/152/87   0.5067   0.4962
0.8159
  11   rs1102      ALL    2    1           34/249/317    0.415   0.3888
0.115
  11   rs1102      AFF    2    1           15/127/158   0.4233   0.3864
0.1339
  11   rs1102    UNAFF    2    1           19/122/159   0.4067   0.3911
0.5565
  11   rs1103      ALL    2    1          126/309/165    0.515   0.4979
0.4136
  11   rs1103      AFF    2    1            57/159/84     0.53    0.496
0.2943
  11   rs1103    UNAFF    2    1            69/150/81      0.5   0.4992
1
```

Importing PLINK-Format Data into R

Importing data in PLINK (LINKAGE) format into R can be tricky. A helpful approach is to create tab-separated text files, where columns are separated by a single, well-defined tab character (`\t`). PLINK genotypes normally span two columns (one per allele); since both belong to the same variable, a different separator between the two allele columns — a space instead of a tab — is preferable. Adding the `--tab` flag produces a text file that can be easily read into R with `read.table(..., sep="\t")`.

In [18]:
```
# Run the plink command
plink  --ped ${output_dir}/cleaned.ped  --map ${output_dir}/cleaned.map  --c
       --tab
```

```
PLINK v1.9.0-b.8 64-bit (22 Oct 2024)             cog-genomics.org/plink/1.
9/
(C) 2005-2024 Shaun Purcell, Christopher Chang   GNU General Public License
v3
Logging to ./output/intro_plink_R/cleaned.R.log.
Options in effect:
  --map ./output/intro_plink_R/cleaned.map
  --out ./output/intro_plink_R/cleaned.R
  --ped ./output/intro_plink_R/cleaned.ped
  --recode
  --tab

Note: --tab flag deprecated.  Use "--recode tab ...".
7836 MB RAM detected; reserving 3918 MB for main workspace.
.ped scan complete (for binary autoconversion).
Performing single-pass .bed write (18 variants, 600 people).
--file: ./output/intro_plink_R/cleaned.R-temporary.bed +
./output/intro_plink_R/cleaned.R-temporary.bim +
./output/intro_plink_R/cleaned.R-temporary.fam written.
18 variants loaded from .bim file.
600 people (329 males, 271 females) loaded from .fam.
600 phenotype values loaded from .fam.
Using 1 thread (no multithreaded calculations invoked).
Before main variant filters, 600 founders and 0 nonfounders present.
Calculating allele frequencies... done.
Total genotyping rate is exactly 1.
18 variants and 600 people pass filters and QC.
Among remaining phenotypes, 300 are cases and 300 are controls.
--recode ped to ./output/intro_plink_R/cleaned.R.ped +
./output/intro_plink_R/cleaned.R.map ... done.
```

> **Note:** PLINK can create a large number of files and will overwrite existing ones without warning, which can get confusing — and is a common source of errors when it's unclear which filtering criteria were actually applied to a given data set before an analysis. Planning your filtering/exporting steps ahead of time, and using clear file names and separate output subdirectories, is highly recommended.

III. Binary PLINK Format

Genome-wide marker genotype data can be massive, resulting in very large file sizes. To reduce file size and speed up calculations, genotype information is usually compressed. Let's convert the text files into binary PLINK format:

In [19]:
```
# Run the plink command
plink  --ped ./data/dbp.cc.ped  --map ./data/dbp.map  --make-bed  --out ${ou
plink  --ped ./data/dbp.qt.ped  --map ./data/dbp.map  --make-bed  --out ${ou
```

```
PLINK v1.9.0-b.8 64-bit (22 Oct 2024)             cog-genomics.org/plink/1.
9/
(C) 2005-2024 Shaun Purcell, Christopher Chang   GNU General Public License
v3
Logging to ./output/intro_plink_R/dbp.log.
Options in effect:
  --make-bed
  --map ./data/dbp.map
  --out ./output/intro_plink_R/dbp
  --ped ./data/dbp.cc.ped

7836 MB RAM detected; reserving 3918 MB for main workspace.
.ped scan complete (for binary autoconversion).
Performing single-pass .bed write (20 variants, 600 people).
--file: ./output/intro_plink_R/dbp-temporary.bed +
./output/intro_plink_R/dbp-temporary.bim +
./output/intro_plink_R/dbp-temporary.fam written.
20 variants loaded from .bim file.
600 people (329 males, 271 females) loaded from .fam.
600 phenotype values loaded from .fam.
Using 1 thread (no multithreaded calculations invoked).
Before main variant filters, 600 founders and 0 nonfounders present.
Calculating allele frequencies... done.
Total genotyping rate is 0.988333.
20 variants and 600 people pass filters and QC.
Among remaining phenotypes, 300 are cases and 300 are controls.
--make-bed to ./output/intro_plink_R/dbp.bed + ./output/intro_plink_R/dbp.bi
m +
./output/intro_plink_R/dbp.fam ... done.
PLINK v1.9.0-b.8 64-bit (22 Oct 2024)             cog-genomics.org/plink/1.
9/
(C) 2005-2024 Shaun Purcell, Christopher Chang   GNU General Public License
v3
Logging to ./output/intro_plink_R/dbp.qt.log.
Options in effect:
  --make-bed
  --map ./data/dbp.map
  --out ./output/intro_plink_R/dbp.qt
  --ped ./data/dbp.qt.ped

7836 MB RAM detected; reserving 3918 MB for main workspace.
.ped scan complete (for binary autoconversion).
Performing single-pass .bed write (20 variants, 600 people).
--file: ./output/intro_plink_R/dbp.qt-temporary.bed +
./output/intro_plink_R/dbp.qt-temporary.bim +
./output/intro_plink_R/dbp.qt-temporary.fam written.
20 variants loaded from .bim file.
600 people (329 males, 271 females) loaded from .fam.
600 phenotype values loaded from .fam.
Using 1 thread (no multithreaded calculations invoked).
Before main variant filters, 600 founders and 0 nonfounders present.
Calculating allele frequencies... done.
Total genotyping rate is 0.988333.
20 variants and 600 people pass filters and QC.
Phenotype data is quantitative.
--make-bed to ./output/intro_plink_R/dbp.qt.bed +
```

```
./output/intro_plink_R/dbp.qt.bim + ./output/intro_plink_R/dbp.qt.fam ... do
ne.
```

PLINK has created four files: `dbp.log` contains all screen output, `dbp.fam` contains the family information, `dbp.bim` the marker information, and `dbp.bed` the marker genotypes in binary (compressed) form. Preview a sample of the `.fam` and `.bim` files below.

In [20]:
```
# Preview the output - dbp.fam
head ${output_dir}/dbp.fam
```

```
4928 1 0 0 1 2
1838 1 0 0 1 2
2450 1 0 0 1 2
647 1 0 0 2 2
2772 1 0 0 1 2
148 1 0 0 2 2
1 1 0 0 1 2
1696 1 0 0 2 2
890 1 0 0 1 2
1832 1 0 0 1 2
```

In [21]:
```
# Preview the output - dbp.bim
head ${output_dir}/dbp.bim
```

```
11      rs1101  0       1021    1       2
11      rs1102  0       3886    2       1
11      rs1103  0       15023   2       1
11      rs1104  0       15788   1       2
11      rs1105  0       21702   1       2
11      rs1106  0       23505   2       1
11      rs1107  0       23508   2       1
11      rs1108  0       28769   2       1
11      rs1109  0       31385   1       2
11      rs1110  0       33198   1       2
```

Part 2: R

> **Kernel:** run the code cells in this section with the **R** kernel.

Starting R

If you're unsure how to use a function, or want to see its full set of arguments, use the `help()` function and `?` help operator in R to access the documentation pages for R functions, data sets, and other objects.

References

- R Core Team (2025), R: A Language and Environment for Statistical Computing, R Foundation for Statistical Computing

I. Data Types

Data can be of different types — for example numeric, string, or logical values. Suppose we want to compile a short list of European cities with a few features for each.

In [1]:
```
# I: Data Types
city       = c("Oslo", "Bergen", "Munich", "Berlin", "Rome", "Milan")
population = c(0.58, 0.25, 1.3, 3.4, 2.7, 1.3)
country    = factor( c("Norway" , "Norway", "Germany",
                       "Germany", "Italy", "Italy"   ))
capital    = c(TRUE, FALSE, FALSE, TRUE, TRUE, FALSE)
updated    = 2009
```

You've now created several data objects — city names, population sizes, countries, capital-city status, and year of last update — in R's working memory using the assignment operator `=`. To print an object's contents, simply type its name:

In [2]:
```
# Return the contents of the objects
city
population
country
capital
```

'Oslo' · 'Bergen' · 'Munich' · 'Berlin' · 'Rome' · 'Milan'

0.58 · 0.25 · 1.3 · 3.4 · 2.7 · 1.3

Norway · Norway · Germany · Germany · Italy · Italy

▶ **Levels**:

TRUE · FALSE · FALSE · TRUE · TRUE · FALSE

Each of these objects is a **vector** — all its elements share the same data type. For example, `city` contains only strings (characters), while `capital` contains only logical values indicating whether each city is its country's capital. Vectors can be concatenated with `c()`, and `summary()` gives a short summary of an object — what it does depends on the object's data type (its *class* in R); for instance, a mean can be calculated for numeric variables but not for nominal ones (represented as the `factor` type in R). An object's type can be checked with various functions. Data type is one attribute of an object, but objects can have more than one attribute — another example is **length**, the number of elements in the object (i.e. the number of entries in a vector):

In [3]:
```
# Concatenate the vectors
c(city, city)
c(population, updated)
```

```
# Get a summary of the objects
summary (city)
summary (population)
summary (country)
summary (capital)

# Assess the type of object
is.numeric(city)
is.character(city)
is.factor(city)

class (city)
class (population)
class (country)
class (capital)

# Return the length (the number of elements of an object)
length(city)
```

'Oslo' · 'Bergen' · 'Munich' · 'Berlin' · 'Rome' · 'Milan' · 'Oslo' · 'Bergen' · 'Munich' · 'Berlin' · 'Rome' · 'Milan'

0.58 · 0.25 · 1.3 · 3.4 · 2.7 · 1.3 · 2009

```
    Length     Class      Mode
         6 character character
    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   0.250   0.760   1.300   1.588   2.350   3.400
```

Germany: 2 **Italy:** 2 **Norway:** 2

```
    Mode   FALSE    TRUE
 logical       3       3
```

FALSE

TRUE

FALSE

'character'

'numeric'

'factor'

'logical'

6

II. Names & Indexes

For better data organization, access, and presentation, elements in a vector can have names. In many analyses you'll want to access only part of a data set, or even a single element.

For example, markers should often be tested for deviations from Hardy-Weinberg equilibrium (HWE) separately in affected and unaffected samples. Elements of data

objects are accessed via **indexes**, of which there are three kinds. The simplest is addressing by position, shown in the code block below.

If an object's elements have names, those names can also be used to access them. A third option for vectors is a **logical index**, where only elements marked `TRUE` are accessed — for example, selecting only the capitals from our list of cities:

In [4]:
```
# II: Names & Indexes #

# Return the name of the elements in a vector
names(population) = city
population

# Access elements of objects by position
city [3]
city [2:4]
city[c(1,5:6)]
population[3]

# Access element of an object by names
population["Oslo"]
population[c("Berlin","Rome")]

# Access elements using a logical index
population
capital
population[capital]
```

Oslo: 0.58 **Bergen:** 0.25 **Munich:** 1.3 **Berlin:** 3.4 **Rome:** 2.7 **Milan:** 1.3

'Munich'

'Bergen' · 'Munich' · 'Berlin'

'Oslo' · 'Rome' · 'Milan'

Munich: 1.3
Oslo: 0.58
Berlin: 3.4 **Rome:** 2.7

Oslo: 0.58 **Bergen:** 0.25 **Munich:** 1.3 **Berlin:** 3.4 **Rome:** 2.7 **Milan:** 1.3

TRUE · FALSE · FALSE · TRUE · TRUE · FALSE

Oslo: 0.58 **Berlin:** 3.4 **Rome:** 2.7

Logical indexes are powerful: you can formulate a condition, store the result as a logical vector, and reuse that vector to repeatedly access only the elements meeting the condition. For example, the following commands select only the cities with a population of at least one million:

In [5]:
```
# Select only those cities from our list which have a population of at least
population>=1.0
population[population>=1.0]
```

Oslo: FALSE **Bergen:** FALSE **Munich:** TRUE **Berlin:** TRUE **Rome:** TRUE **Milan:** TRUE

Munich: 1.3 **Berlin:** 3.4 **Rome:** 2.7 **Milan:** 1.3

III. Data Frames

Data sets are often presented in tabular form: columns usually represent features or measurements and share a single data type, while rows represent observations or samples and may mix different data types. Worksheets from SPSS or Excel, and tables extracted from SQL databases, typically follow this format.

Data frames are special lists where all vectors (features/columns) have identical length, plus added functionality for printing, summarizing, and so on. They have two dimensions — rows and columns — and both can have names.

In [6]:
```
# III: Data frames

# Representation of a data frame
cities = data.frame (city=city, pop=population,
                     country=country, capital=capital, stringsAsFactors=F)
cities
length(cities)
dim(cities)
is.data.frame(cities)
is.list(cities)

# Retrieve column names of the data frame
colnames(cities)
# Retrieve row names of the data frame
rownames(cities)
```

A data.frame: 6 × 4

	city	pop	country	capital
	<chr>	<dbl>	<fct>	<lgl>
Oslo	Oslo	0.58	Norway	TRUE
Bergen	Bergen	0.25	Norway	FALSE
Munich	Munich	1.30	Germany	FALSE
Berlin	Berlin	3.40	Germany	TRUE
Rome	Rome	2.70	Italy	TRUE
Milan	Milan	1.30	Italy	FALSE

4

6 · 4

TRUE

TRUE

'city' · 'pop' · 'country' · 'capital'

'Oslo' · 'Bergen' · 'Munich' · 'Berlin' · 'Rome' · 'Milan'

Indexing works similarly to vectors, but since data frames have two dimensions (rows and columns), we need two indexes. We can access single elements, or entire rows or columns.

In [7]:
```
# Index the data frame using logical indexes
cities$city
cities[,1]
cities[2,]
cities[2,3]
cities$pop[3]
cities[capital,]
cities[cities$pop>=1.0,]
```

'Oslo' · 'Bergen' · 'Munich' · 'Berlin' · 'Rome' · 'Milan'

'Oslo' · 'Bergen' · 'Munich' · 'Berlin' · 'Rome' · 'Milan'

A data.frame: 1 × 4

	city	pop	country	capital
	<chr>	<dbl>	<fct>	<lgl>
Bergen	Bergen	0.25	Norway	FALSE

Norway

▶ **Levels**:

1.3

A data.frame: 3 × 4

	city	pop	country	capital
	<chr>	<dbl>	<fct>	<lgl>
Oslo	Oslo	0.58	Norway	TRUE
Berlin	Berlin	3.40	Germany	TRUE
Rome	Rome	2.70	Italy	TRUE

A data.frame: 4 × 4

	city	pop	country	capital
	<chr>	<dbl>	<fct>	<lgl>
Munich	Munich	1.3	Germany	FALSE
Berlin	Berlin	3.4	Germany	TRUE
Rome	Rome	2.7	Italy	TRUE
Milan	Milan	1.3	Italy	FALSE

IV. Export & Import

All R objects live in working memory, and are lost when you quit R unless you've saved them to disk. There are several ways to save objects. It's often useful to export a data set to text format so it can be opened in a text editor (or Word) or imported into other software — for data frames, this can be done with `write.table()`.

In [8]:
```
# IV: Export & Import

# Get an overview on which objects are currently held in the working memory:
ls()

# Save the objects cities, city, and country in an external archive file cal
# Note that this file is in a format that is only readable with R!
save(cities, city, country, file="output/intro_plink_R/myobjects.R")

# Export the data set to text format
write.table(cities, file="output/intro_plink_R/cities.txt")

# Re-direct output from R console to a text file
sink ("output/intro_plink_R/cities.output.txt")
print (cities)
sink ()

# Check if these files have properly been created in the working directory c
dir()
```

'capital' · 'cities' · 'city' · 'country' · 'population' · 'updated'

'data' · 'Intro_Plink_R.ipynb' · 'output' · 'Regression.ipynb'

The `dir()` command lists the contents of your current working directory. If `cities.txt` and `cities.output.txt` weren't created, check your script for errors or ask for help. Once they've been created, delete all objects from R's working memory.

In [9]:
```
# Delete all objects from R working memory
rm(list=ls())
ls()

# Import the data frame from the external text file
new.table = read.table ("./output/intro_plink_R/cities.txt")
ls()
new.table

# Import the objects from the R archive file
load ("output/intro_plink_R/myobjects.R")
ls()
cities
new.table
```

'new.table'

A data.frame: 6 × 4

	city	pop	country	capital
	<chr>	<dbl>	<chr>	<lgl>
Oslo	Oslo	0.58	Norway	TRUE
Bergen	Bergen	0.25	Norway	FALSE
Munich	Munich	1.30	Germany	FALSE
Berlin	Berlin	3.40	Germany	TRUE
Rome	Rome	2.70	Italy	TRUE
Milan	Milan	1.30	Italy	FALSE

'cities' · 'city' · 'country' · 'new.table'

A data.frame: 6 × 4

	city	pop	country	capital
	<chr>	<dbl>	<fct>	<lgl>
Oslo	Oslo	0.58	Norway	TRUE
Bergen	Bergen	0.25	Norway	FALSE
Munich	Munich	1.30	Germany	FALSE
Berlin	Berlin	3.40	Germany	TRUE
Rome	Rome	2.70	Italy	TRUE
Milan	Milan	1.30	Italy	FALSE

A data.frame: 6 × 4

	city	pop	country	capital
	<chr>	<dbl>	<chr>	<lgl>
Oslo	Oslo	0.58	Norway	TRUE
Bergen	Bergen	0.25	Norway	FALSE
Munich	Munich	1.30	Germany	FALSE
Berlin	Berlin	3.40	Germany	TRUE
Rome	Rome	2.70	Italy	TRUE
Milan	Milan	1.30	Italy	FALSE